Operating Systems and System Programming

Skill-1

Aim

To set up a Linux development environment by installing a Linux VM, configuring GCC, creating a Git repository, organizing the project structure, understanding shell architecture, and building an initial Makefile, and to analyze process abstraction by executing fork(), understanding the exec() family, studying parent-child relationships, inspecting the process tree, and practicing system-call tracing.

Algorithm

1. Install and configure the Linux virtual machine.

2. Install and verify the GCC compiler.

3. Create a project directory with an appropriate structure.

4. Initialize a Git repository and track the project files.

5. Study the basic architecture and working of the Linux shell.

6. Create an initial Makefile to compile and manage the C programs.

7. Develop a C program using fork() to create parent and child processes.

8. Display the PID and PPID of the parent and child processes.

9. Study the exec() family of system calls and observe how a process executes a new program.

10. Analyze the relationship between parent and child processes.

11. Inspect the process hierarchy using Linux commands such as ps and pstree.

12. Use strace to trace and analyze the system calls made by the programs.

13. Record and analyze the observations.

14. Stop the experiment.

Task-1:

Step-1: Check Linux version

```
allur@AKSHAYA:~$ uname -a
Linux AKSHAYA 6.18.33.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun 18 21:54:43 UTC 202
6 x86_64 GNU/Linux
allur@AKSHAYA:~$
```

Step-2: Install GCC

```
allur@AKSHAYA:~$ sudo apt update
[sudo: authenticate] Password:
Get:1 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]
Hit:2 http://archive.ubuntu.com/ubuntu resolute InRelease
Get:3 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Packages [417 kB]
Get:4 http://archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]
Get:5 http://security.ubuntu.com/ubuntu resolute-security/main Translation-en [97.2 kB]
Get:6 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Components [46.7 kB]
Get:7 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Packages [159 kB]
Get:8 http://security.ubuntu.com/ubuntu resolute-security/universe Translation-en [51.2 kB]
Get:9 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Components [43.1 kB]
Get:10 http://security.ubuntu.com/ubuntu resolute-security/restricted amd64 Packages [349 kB]
Get:11 http://security.ubuntu.com/ubuntu resolute-security/restricted Translation-en [67.8 kB]
Hit:12 http://archive.ubuntu.com/ubuntu resolute-backports InRelease
Get:13 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 Packages [492 kB]
Get:14 http://archive.ubuntu.com/ubuntu resolute-updates/main Translation-en [118 kB]
Get:15 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 Components [97.8 kB]
Get:16 http://archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Packages [244 kB]
Get:17 http://archive.ubuntu.com/ubuntu resolute-updates/universe Translation-en [81.0 kB]
Get:18 http://archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Components [184 kB]
Get:19 http://archive.ubuntu.com/ubuntu resolute-updates/restricted amd64 Packages [367 kB]
Get:20 http://archive.ubuntu.com/ubuntu resolute-updates/restricted Translation-en [70.8 kB]
Fetched 3159 kB in 3s (1092 kB/s)
30 packages can be upgraded. Run 'apt list --upgradable' to see them.
allur@AKSHAYA:~$
```

```
allur@AKSHAYA:~$ sudo apt install gcc
gcc is already the newest version (4:15.2.0-5ubuntu1).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 30
allur@AKSHAYA:~$
```

```
allur@AKSHAYA:~$ gcc --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

allur@AKSHAYA:~$
```

Step-3: Install Git

```
allur@AKSHAYA:~$ sudo apt install git
git is already the newest version (1:2.53.0-1ubuntu1).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 30
allur@AKSHAYA:~$
```

```
allur@AKSHAYA:~$ git --version
git version 2.53.0
allur@AKSHAYA:~$
```

Step-4: Create project directory

```
allur@AKSHAYA:~$ mkdir skill1
allur@AKSHAYA:~$
```

```
allur@AKSHAYA:~$ cd skill1
allur@AKSHAYA:~/skill1$
```

```
allur@AKSHAYA:~/skill1$ mkdir src include bin
allur@AKSHAYA:~/skill1$
```

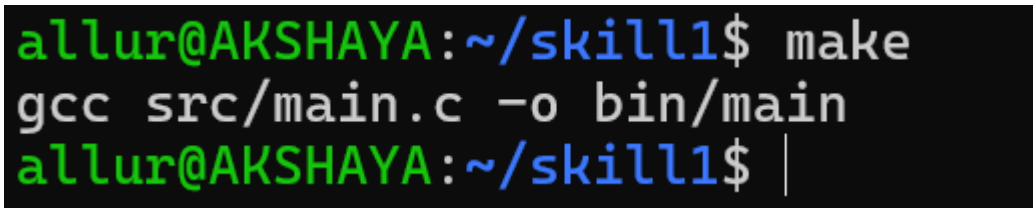
Step-5: Initialize Git repository

```
allur@AKSHAYA:~/skill1$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: will change to "main" in Git 3.0. To configure the initial branch name
hint: to use in all of your new repositories, which will suppress this warning,
hint: call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
hint:
hint: Disable this message with "git config set advice.defaultBranchName false"
Initialized empty Git repository in /home/allur/skill1/.git/
```

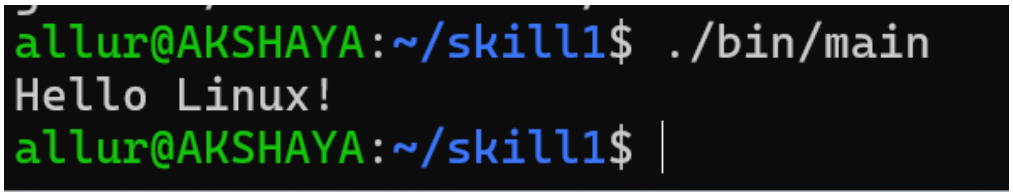
Step-6: Create a simple C program

```
allur@AKSHAYA:~/skill1$ nano src/main.c
```

Step-8: Build the program

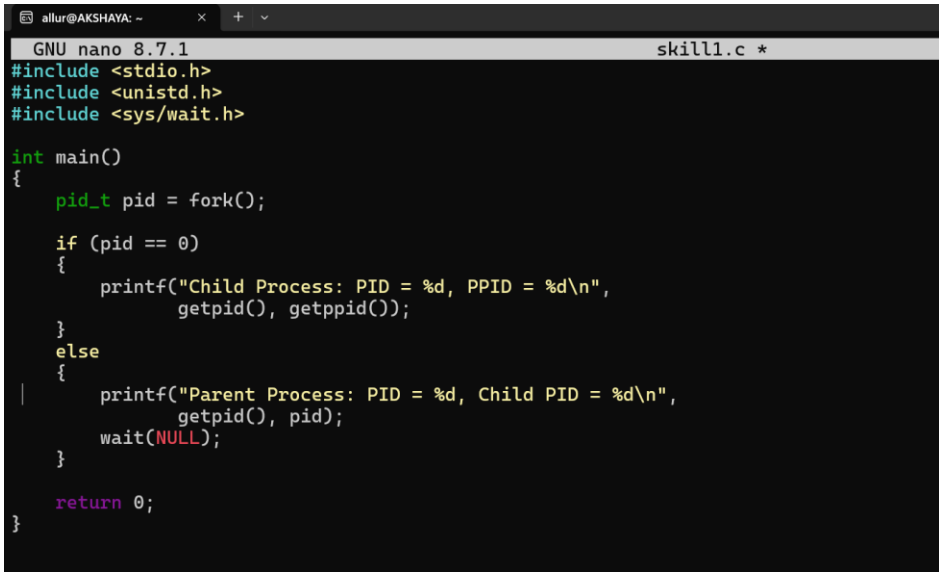


Step-9: Run the program

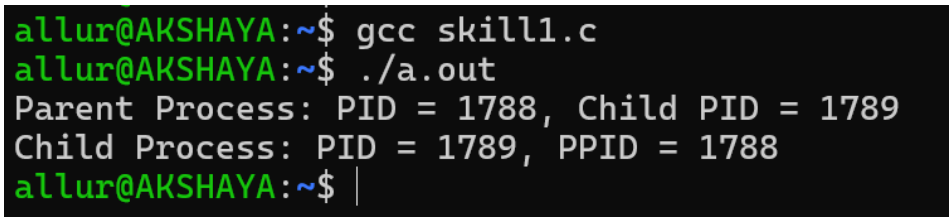


Task-2:

Step-1: Create a process using fork()

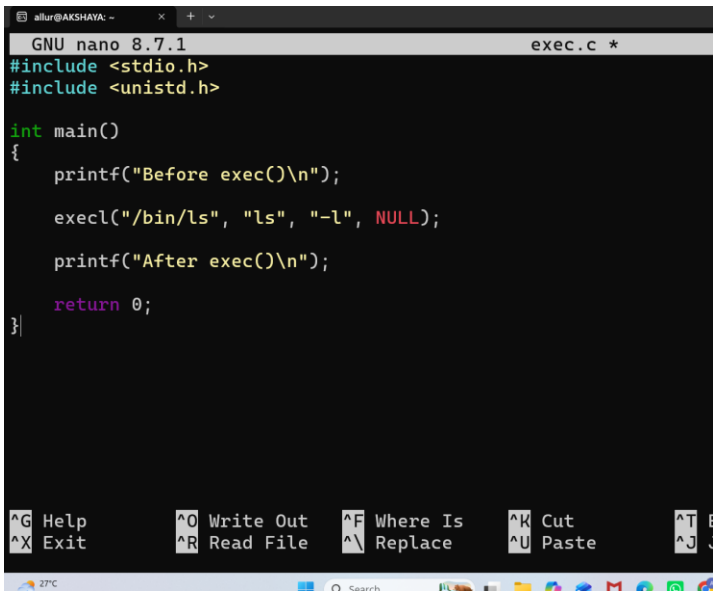


Step-2: Compile and run



Step-3: Understand exec()

The exec() family replaces the current process with another program.



```
allur@AKSHAYA:~$ gcc exec.c
```

```
allur@AKSHAYA:~$ ./a.out
Before exec()
total 148
-rwxr-xr-x 1 allur allur 15992 Aug 23 06:40 a.out
-rw-r--r-- 1 allur allur  1004 Jul 30 05:31 command
-rw------- 1 allur allur  1004 Jul 30 05:27 command.save
-rw------- 1 allur allur  1004 Jul 30 05:28 command.save.1
-rw------- 1 allur allur  1004 Jul 30 05:29 command.save.2
-rw-r--r-- 1 allur allur  1004 Jul 30 05:32 command_exec.c
-rw------- 1 allur allur  1004 Jul 30 05:30 command_exec.c.save
-rw-r--r-- 1 allur allur    43 Aug 22 07:38 destination.txt
-rw-r--r-- 1 allur allur   173 Aug 23 06:39 exec.c
-rw-r--r-- 1 allur allur   157 Aug 11 14:28 fork.c
-rwxr-xr-x 1 allur allur 16256 Aug 18 05:52 fork_demo
-rw-r--r-- 1 allur allur   818 Aug 18 06:08 fork_demo.c
-rw-r--r-- 1 allur allur   623 Aug 22 07:37 practical-2.c
-rw-r--r-- 1 allur allur   815 Aug 22 14:53 practical3.c
-rwxr-xr-x 1 allur allur 16472 Jul 30 05:34 process
-rw-r--r-- 1 allur allur  1004 Jul 30 05:34 process.c
-rw------- 1 allur allur  1004 Jul 30 05:26 process.c.save
-rwxr-xr-x 1 allur allur 16000 Aug 11 15:05 read
-rw-r--r-- 1 allur allur    75 Aug 11 15:05 read.c
drwxr-xr-x 6 allur allur  4096 Aug 22 16:03 skill1
drwxr-xr-x 2 allur allur  4096 Aug  8 10:11 skill1.1
-rw-r--r-- 1 allur allur     0 Aug 22 15:49 skill1.1.c
-rw-r--r-- 1 allur allur   373 Aug 23 06:39 skill1.c
-rw-r--r-- 1 allur allur    43 Aug 22 07:33 source.txt
-rw-r--r-- 1 allur allur  1004 Aug  1 09:34 task-1.c
allur@AKSHAYA:~$
```

Step-4:Check Process Tree

```
allur@AKSHAYA:~$ pstree
systemd─┬─agetty
        ├─chronyd-starter───chronyd───chronyd
        ├─cron
        ├─dbus-daemon
        ├─init-systemd(Ub─┬─SessionLeader───Relay(404)───bash───pstree
        │                 ├─init───{init}
        │                 ├─login───bash
        │                 └─{init-systemd(Ub}
        ├─networkd-dispat
        ├─rsyslogd───3*[{rsyslogd}]
        ├─systemd───(sd-pam)
        ├─systemd-journal
        ├─systemd-logind
        ├─systemd-resolve
        ├─systemd-udevd
        ├─unattended-upgr───{unattended-upgr}
        └─wsl-pro-service───7*[{wsl-pro-service}]
allur@AKSHAYA:~$
```

Step-5:Trace System Calls

```
allur@AKSHAYA:~$ gcc fork.c -o fork
allur@AKSHAYA:~$ ./fork
Hai
Hello
```

```
allur@AKSHAYA:~$ strace ./fork
execve("./fork", ["./fork"], 0x7ffd03004490 /* 25 vars */) = 0
brk(NULL)                               = 0x6304d11bd000
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7908ea807000
access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=20087, ...}) = 0
mmap(NULL, 20087, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7908ea802000
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0 \250\2\0\0\0\0\0"..., 832) = 832
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 840, 64) = 840
fstat(3, {st_mode=S_IFREG|0755, st_size=2186512, ...}) = 0
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 840, 64) = 840
mmap(NULL, 2231696, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7908ea400000
mmap(0x7908ea428000, 1671168, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x7908ea428000
mmap(0x7908ea5c0000, 319488, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1c0000) = 0x7908ea5c0000
mmap(0x7908ea60e000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x20d000) = 0x7908ea60e000
mmap(0x7908ea614000, 52624, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7908ea614000
close(3)                                = 0
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7908ea7ff000
arch_prctl(ARCH_SET_FS, 0x7908ea7ff740) = 0
set_tid_address(0x7908ea7ffd68)         = 776
set_robust_list(0x7908ea7ffa20, 24)     = 0
rseq(0x7908ea7ff6a0, 0x20, 0, 0x53053053) = 0
mprotect(0x7908ea60e000, 16384, PROT_READ) = 0
mprotect(0x63049bec3000, 4096, PROT_READ) = 0
mprotect(0x7908ea84c000, 8192, PROT_READ) = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
getrandom("\x23\x20\xa0\x90\x04\x9c\xac\xb7", 8, GRND_NONBLOCK) = 8
munmap(0x7908ea802000, 20087)           = 0
rt_sigprocmask(SIG_BLOCK, ~[], [], 8)   = 0
clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD, child_tidptr=0x7908ea7ffa10) = 777
Hello
rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0
--- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=777, si_uid=1000, si_status=0, si_utime=0, si_stime=0} ---
fstat(1, {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0), ...}) = 0
brk(NULL)                               = 0x6304d11bd000
brk(0x6304d11de000)                     = 0x6304d11de000
write(1, "Hai\n", 4Hai
)                   = 4
exit_group(0)                           = ?
+++ exited with 0 +++
allur@AKSHAYA:~$
```

Result

The Linux development environment was successfully configured with GCC, Git, project structure, and an initial Makefile. The experiment provided practical understanding of Linux shell architecture and process abstraction, including fork(), the exec() family, parent-child relationships, and process trees. System-call tracing using strace helped demonstrate how userlevel programs interact with the Linux kernel during process creation and execution.